

OptiCrop

Smart Agricultural Production Optimization Engine

Project Development Phase — Document 2: Coding and Solutioning

1. Problem & Solution Approach

Farmers often struggle to decide which crop best suits the current soil and climate conditions on their land. OptiCrop addresses this by using a supervised machine learning model trained on historical agricultural data (Nitrogen, Phosphorous, Potassium, temperature, humidity, pH, and rainfall) to recommend the most suitable crop. The solution is delivered as a Flask web application so that any user can enter their conditions through a browser and instantly receive a recommendation.

The solutioning approach follows five stages: data understanding (EDA), data cleaning (outlier handling), unsupervised pattern discovery (K-Means clustering), supervised prediction (Logistic Regression), and deployment (Flask web app).

2. Exploratory Data Analysis — Univariate Output

Distribution plots were generated for each of the seven numeric features, along with a count plot confirming the dataset is perfectly balanced (100 samples per crop):

```
for col in numeric_cols:
    sns.histplot(df[col], kde=True, bins=30, color='green')
    plt.title(f'Distribution of {col}')
    plt.show()
```

```
sns.countplot(x='label', data=df)
```

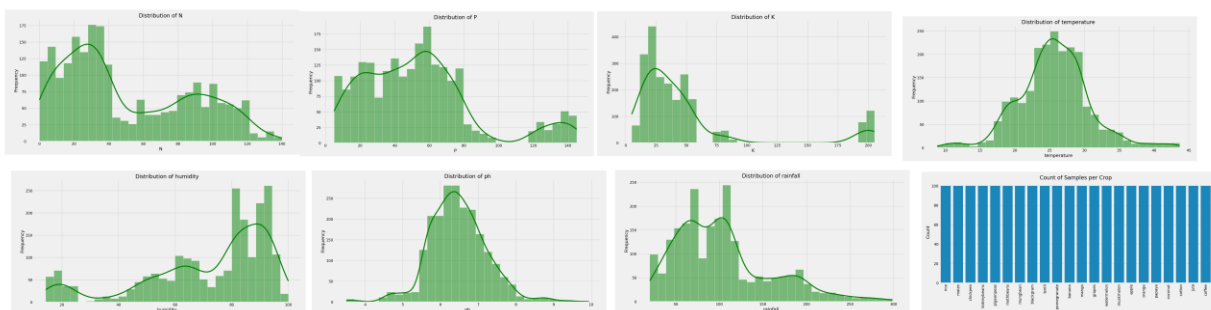
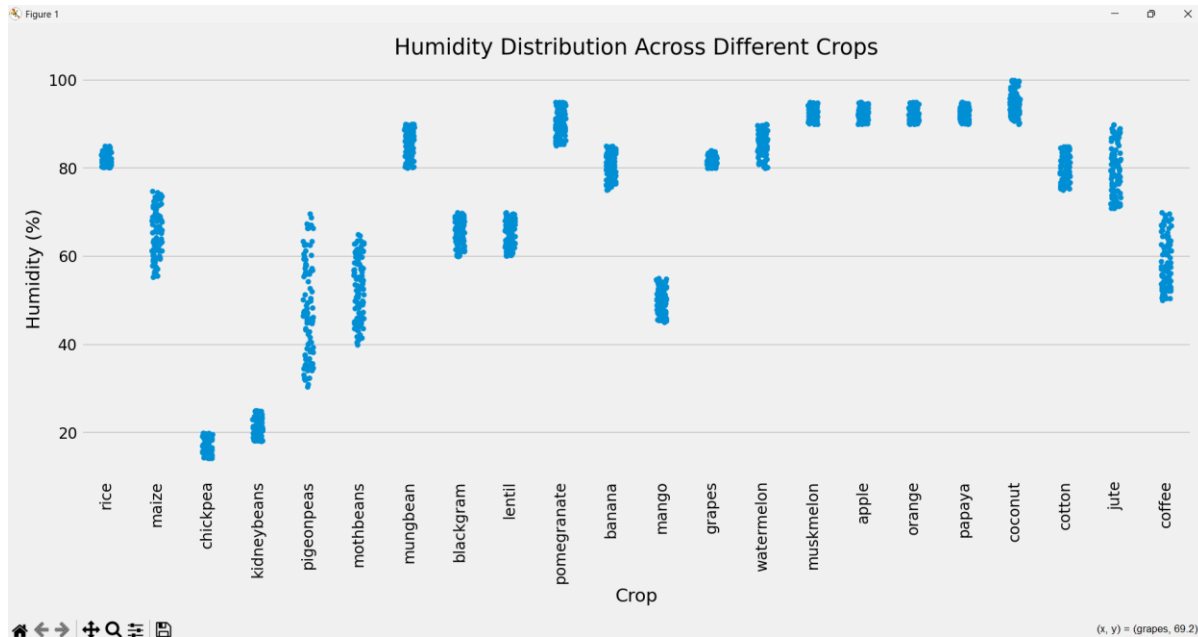


Figure 1: Univariate distribution of N, P, K, temperature, humidity, pH, rainfall, and sample count per crop

Several features (N, K, rainfall) show multimodal distributions — multiple peaks rather than a single bell curve — indicating that different crop groups cluster around distinct nutrient/rainfall ranges, which is a strong early signal that these features will help separate crops effectively.

3. Bivariate Analysis Output — Humidity vs Crop

```
sns.stripplot(x='label', y='humidity', data=df, jitter=True)
plt.title('Humidity Distribution Across Different Crops')
```



This confirms humidity is a strong differentiator: chickpea and kidneybeans cluster near 15-25% humidity, while coconut and several fruit crops require 85-100% humidity.

4. Multivariate Analysis Output

```
df_melted = df.melt(value_vars=numeric_cols, var_name='Feature', value_name='Value')
sns.barplot(x='Feature', y='Value', data=df_melted, estimator=np.mean)
```

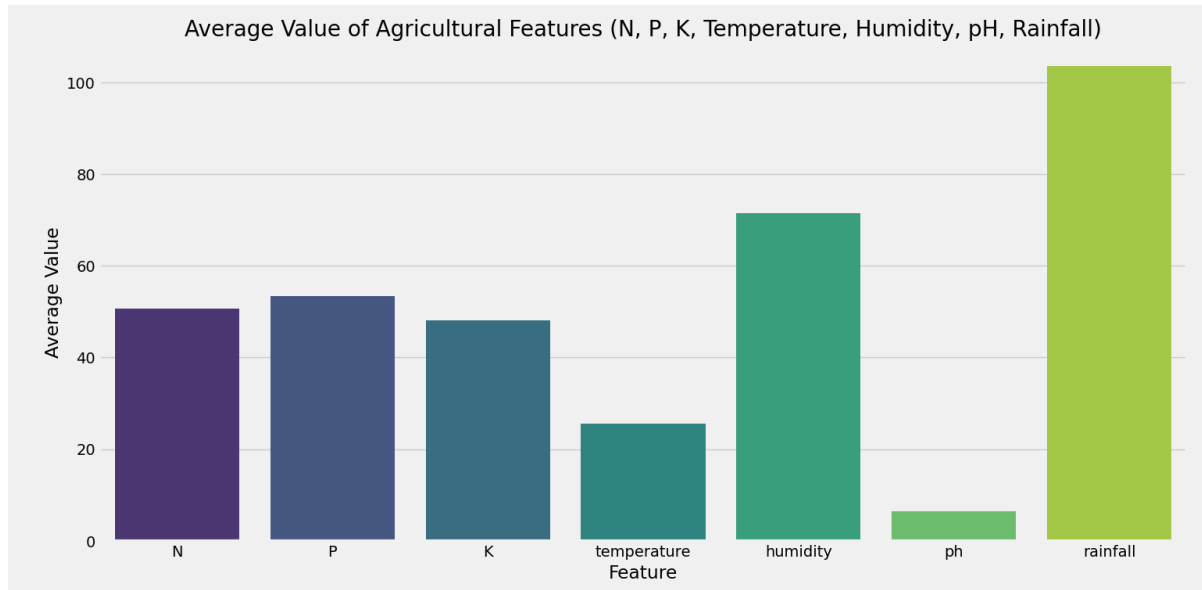


Figure 3: Average value of each agricultural feature across the dataset

5. Data Preprocessing & Outlier Handling

A combined boxplot across all seven numeric features revealed that Potassium had a tight, distinct cluster of outliers around 200, and rainfall, phosphorous, and a few others also showed mild outliers:

```
plt.figure(figsize=(12, 6))
sns.boxplot(data=df[numeric_cols])
plt.title('Boxplot of All Features - Outlier Overview')
```

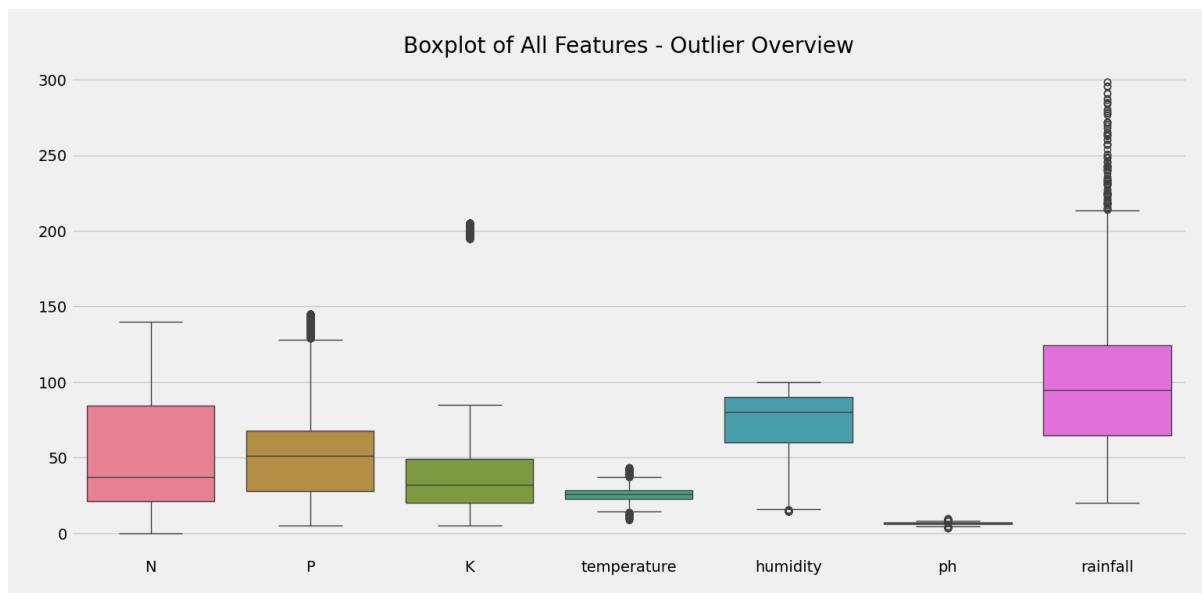


Figure 4: Outlier overview across all features (IQR boxplots)

Per the project methodology, a log transformation (np.log1p) was applied to Potassium to normalize its distribution, and rows with Phosphorous outliers were removed using the IQR filter rule:

```
df['K'] = np.log1p(df['K'])

Q1 = df['P'].quantile(0.25)
Q3 = df['P'].quantile(0.75)
IQR = Q3 - Q1
filter_p = (df['P'] >= Q1 - 1.5 * IQR) & (df['P'] <= Q3 + 1.5 * IQR)
df = df.loc[filter_p]
```

6. Train/Test Split

```
y = df['label']
x = df.drop(['label'], axis=1)

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=0)
```

7. K-Means Clustering Solution

The Elbow Method was used to identify the optimal number of clusters. The WCSS curve shows a clear bend around 4 clusters, after which the rate of decrease flattens significantly:

```
wcss = []
for i in range(1, 11):
    km = KMeans(n_clusters=i, init="k-means++", max_iter=300,
                n_init=10, random_state=0)
    km.fit(x)
    wcss.append(km.inertia_)
plt.plot(range(1, 11), wcss)
```

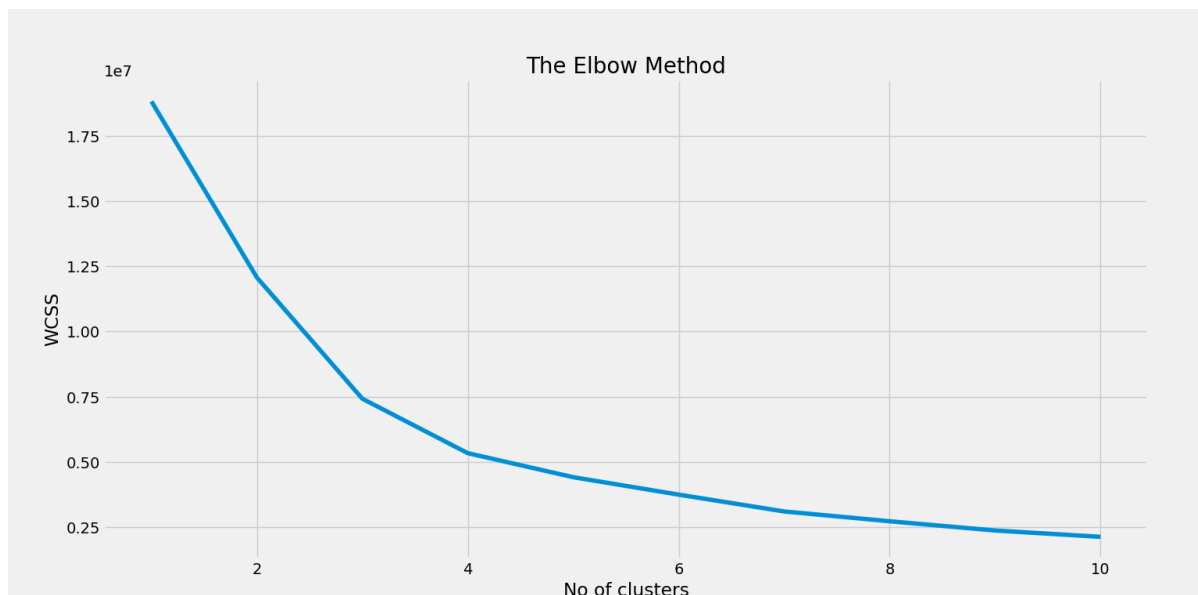


Figure 5: Elbow Method — WCSS vs Number of Clusters

Based on this elbow point, K-Means was trained with 4 clusters to discover natural environmental groupings among the crops:

```
km = KMeans(n_clusters=4, init="k-means++", max_iter=300,
            n_init=10, random_state=0)
y_means = km.fit_predict(x)
```

8. Feature Scaling & Logistic Regression Solution

Since input features span very different numeric ranges (e.g., rainfall 20-300 vs pH ~6), StandardScaler was applied before training the Logistic Regression classifier:

```
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

model = LogisticRegression(max_iter=1000)
model.fit(x_train_scaled, y_train)
y_pred = model.predict(x_test_scaled)
```

9. Model Evaluation Output

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Overall Accuracy: {accuracy:.2f}")
print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='YlGnBu')
```

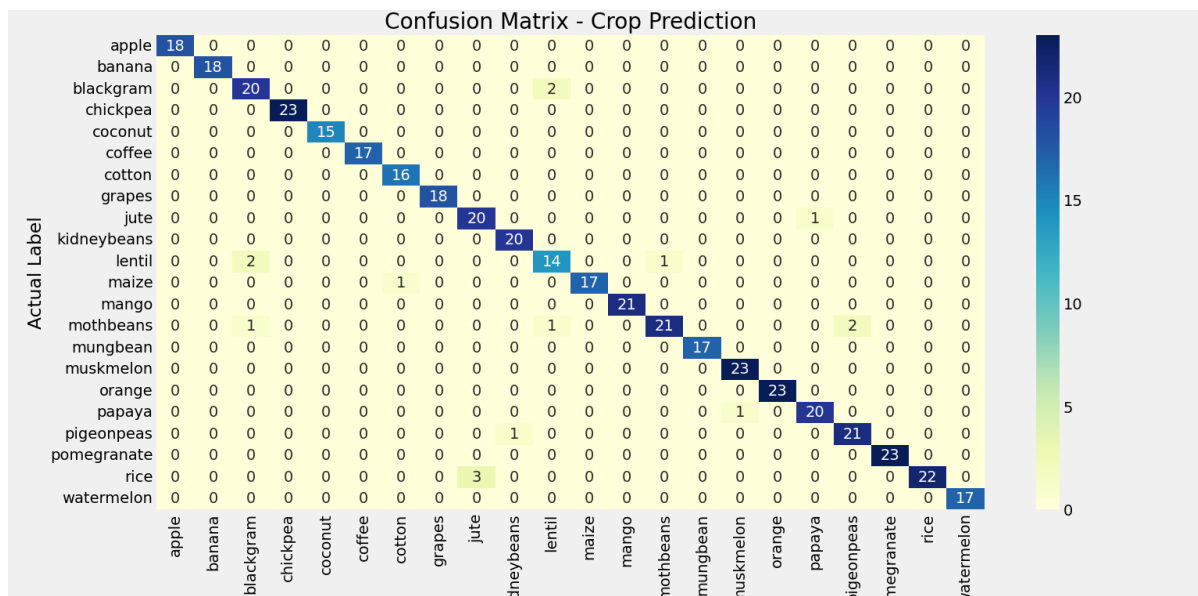


Figure 6: Confusion Matrix — Actual vs Predicted Crop Labels

Result: The model achieved 96% overall accuracy on the held-out test set. The confusion matrix shows a strong, clean diagonal — meaning nearly every crop was predicted correctly. The only minor confusions occur between environmentally similar crops: lentil/blackgram (2-3 samples), jute/rice (a few samples), and mothbeans/pigeonpeas — all crops that share overlapping rainfall and humidity ranges, which is consistent with the bivariate analysis findings in Figure 2.

10. Model Persistence

```
import pickle
with open('model.pkl', 'wb') as model_file:
    pickle.dump(model, model_file)
with open('scaler.pkl', 'wb') as scaler_file:
    pickle.dump(scaler, scaler_file)
```

11. Flask Backend Solution (app.py)

The Flask backend loads the saved model and scaler once at startup, defines navigation routes, and implements the prediction endpoint that reproduces the exact training-time preprocessing before calling `model.predict()`:

```
import numpy as np
import pandas as pd
from flask import Flask, request, render_template
import pickle

app = Flask(__name__)
model = pickle.load(open('model.pkl', 'rb'))
scaler = pickle.load(open('scaler.pkl', 'rb'))

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/findyourcrop')
def findyourcrop():
    return render_template('findyourcrop.html')

@app.route('/predict', methods=['POST'])
def predict():
    int_features = [float(x) for x in request.form.values()]
    input_df = pd.DataFrame([int_features],
        columns=['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall'])
    input_df['K'] = np.log1p(input_df['K'])
    features_scaled = scaler.transform(input_df)
    prediction = model.predict(features_scaled)
    output = prediction[0]
    return render_template('findyourcrop.html',
        prediction_text='Best crop for given conditions is {}'.format(output))

if __name__ == "__main__":
    app.run(debug=True)
```

12. Key Solutioning Decisions

- Consistency between training and inference: the same `log1p` transformation applied to Potassium during training is re-applied to every new user submission before scaling and prediction, preventing data drift between offline analysis and the live app.

- StandardScaler reused via transform(), not fit_transform(), at inference time, so new inputs are scaled using the statistics learned from the training set only.
- Logistic Regression chosen as the final model after evaluation showed strong, well-balanced precision/recall across all 22 crop classes with a simple, fast, and interpretable algorithm suitable for deployment.
- Pickle serialization decouples model training from the web application, so the Flask app starts instantly without needing to retrain on every restart.

13. Conclusion

The combination of careful preprocessing, exploratory analysis, unsupervised clustering for pattern discovery, and a tuned Logistic Regression classifier resulted in a 96% accurate crop recommendation engine, successfully deployed as an interactive Flask web application. The visual outputs above confirm both correct data understanding (EDA) and strong final model performance (confusion matrix).